IIMT3642: Managing and Mining Big Data – Artificial Neural Networks

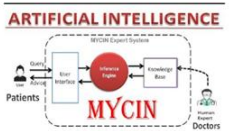
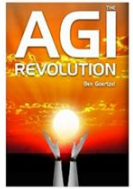
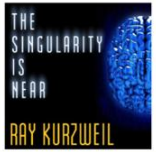
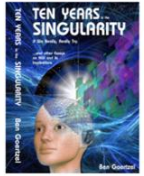
Zhepeng (Lionel) Li, PhD
Associate Professor
Innovation and Information Management

Outline

- Introduction to Artificial Neural Networks and Deep Learning
- Single-layer Artificial Neural Networks (Perceptron)
- Multi-layer Artificial Neural Networks
- Discussion: Modern Deep Learning Model & Technique

A brief history of AI and milestones

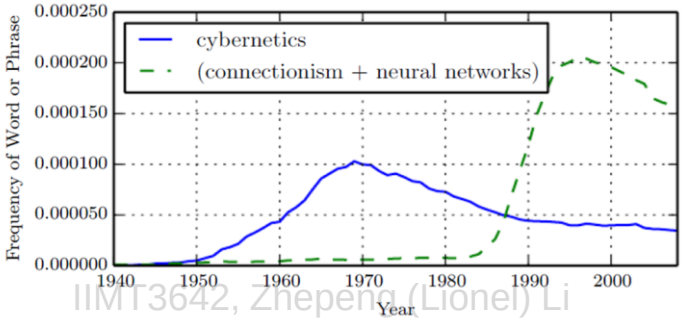
The Four Waves of AI

First Wave	Second Wave	Third Wave	Fourth Wave
c. 1970s - 1990s	*c. 2000s - present*	*est. 2020s - 2030s*	*est. 2030s →*
Good at reasoning, but no ability to learn or generalize. • GOFAI - "Good Old Fashioned AI." • Symbolic, heuristic, rule based. • Handcrafted knowledge, "expert systems."  	Good at learning and perceiving, but minimal ability to reason or generalize. • Statistical learning, "deep" neural nets, CNNs, RNNs. • Advanced text, speech, language and vision processing. 	Excellent at perceiving, learning and reasoning, and able to generalize. • Contextual adaptation, able to explain decisions. • Can converse in natural language. • Requires far fewer data samples for training. • Able to learn and function with minimal supervision. SingularityNET aigo.ai Pandai	Able to perform any intellectual task that a human can. • AGI (Artificial General Intelligence), possibly leading to ASI (Artificial Superintelligence) and the "Technological Singularity."   

Six Kin Development (adapted from DARPA's "Three Waves of AI")

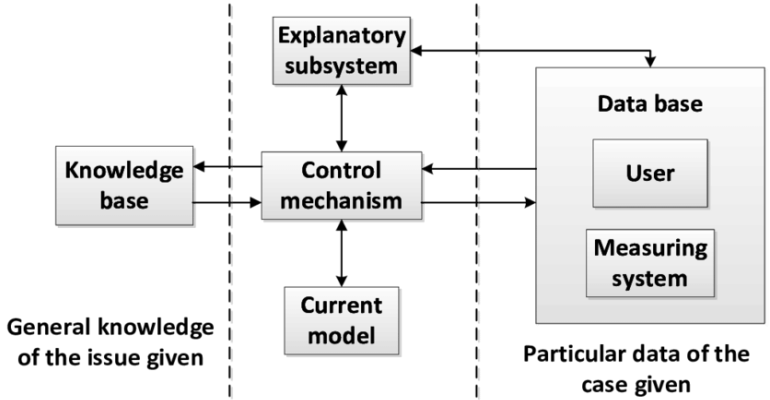
Driving approaches of AI

- Cybernetics — During 1940–1960
  - How to steer a system towards a goal (feedback, control, and regulation)
- Connectionism — During 1980–1990
  - How to build a system that can represent and learn patterns (represent, approximate, and generalize)
- Deep Learning — Since 2006 (too recent to show in the figure)

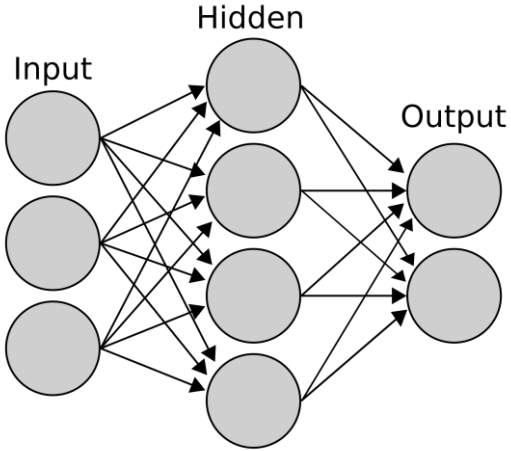


Examples - cybernetics vs. connectionism

- A Schema of Expert System by Cybernetics



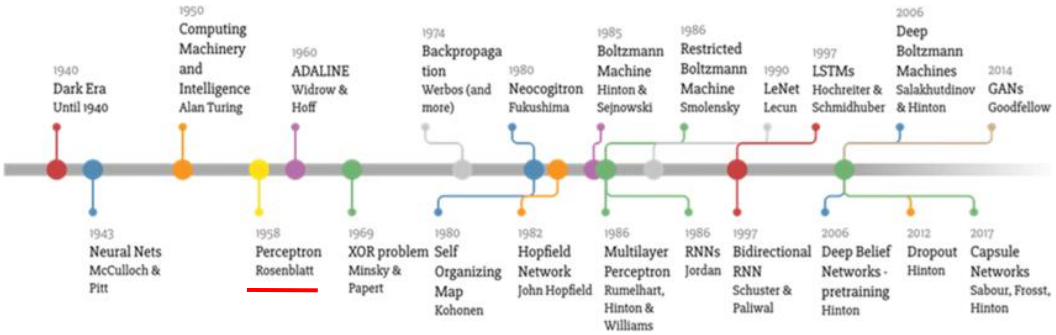
- A Computing Network Structure by Connectionism



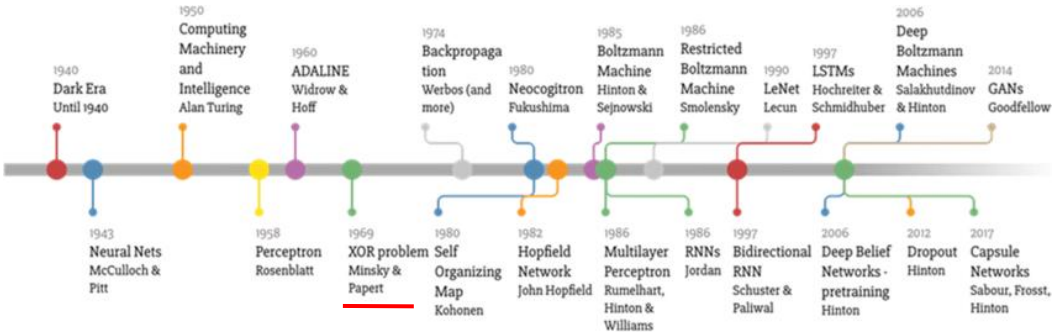
It turns out less is more!

Timeline

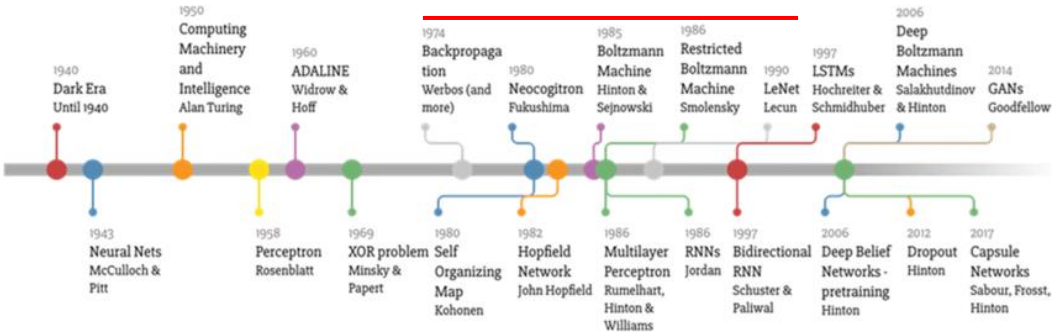
- Perceptron (1958), the first model of a computational unit, was inspired by the biological model of a neuron
  - Was implemented using a large piece of hardware
  - Generated great excitement but failed to live up to inflated expectations



- Pitfall of linear models - The first neural winter: Minsky and Papert's Book
  - Minsky and Papert's book "Perceptrons" (1969) showed that the perceptron only had limited expressive power
  - The book also provided a pessimistic outlook on training multilayer neural networks



- Backpropagation
  - Paul Werbos proposed backpropagation in his 1974 thesis, but was promptly ignored
  - Rumelhart, Hinton, and Williams wrote two papers on backpropagation in 1986
  - Yann LeCun et al. proposed backpropagation for CNN (1989)
  - Backpropagation is more accepted in many works in 1990s, but SVM was the panacea



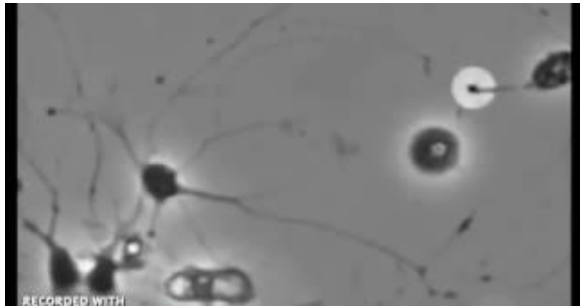
- Modern neural-network architectures are similar to those available in the year 2000 (with some optimization tweaks).
  - Main difference: Lots of data and computational power
  - Possible to train large and deep neural networks with millions of neurons.

... and Next

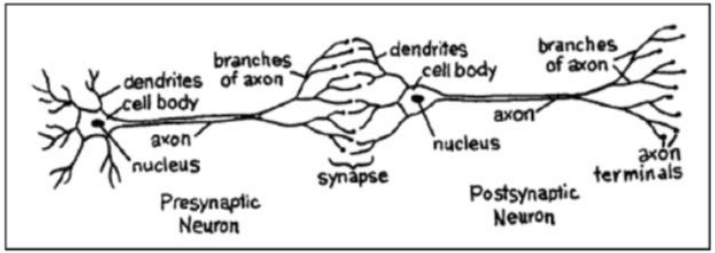
- Significant Events: Crushing victories of deep learning methods in ImageNet competitions (ILSVRC) after 2010.
- Anticipatory Excitement: In a few years, we will have the power to train neural networks with as many computational units as the human brain.
  - The human brain consists of 100 billion neurons, each connecting to 10,000 other neurons.
- Large Models (e.g., Large Language Models)
  - Building up from pre-trained foundation models into more human-like intelligence

Modeling Concepts: Neuron and Perceptron

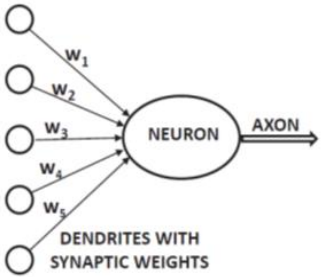
- The Biological Inspiration - When it all began
  - Neural networks were originally proposed to simulate the learning process in biological organisms, such as human nervous system.
  - The human nervous system comprises cells called neurons.
  - The neurons are connected to one another with the use of synapses.
    - The strengths of synaptic connections often change in response to external stimuli.
    - This change causes learning in living organisms.



Perceptron Model



(a) Biological neural network



(b) Artificial neural network

- Neural networks contain computation units ⇒ Neurons.
- The computational units are connected to one another through weights ⇒ Strengths of synaptic connections in biological organisms.
- Each input to a neuron is scaled with a weight, which affects the function computed at that unit.

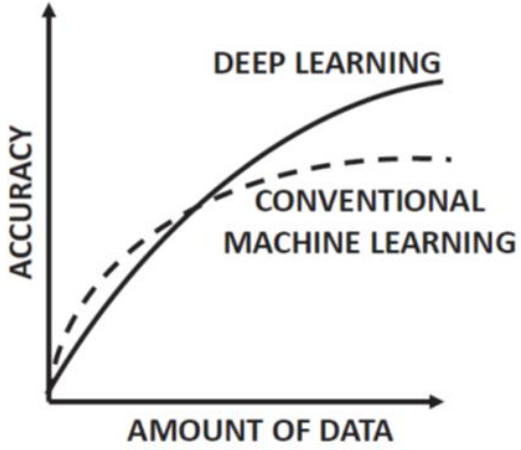
Learning Process – Weight Updating

- In living organisms, synaptic weights change in response to external stimuli.
  - E.g., An unpleasant experience will change the synaptic weights of an organism, which will train the organism to behave differently.

- In artificial neural networks, the weights are learned with the use of training data, which are input-output pairs (e.g., images and their labels).
  - An error made in predicting the label of an image is the unpleasant “stimulus” that changes the weights of the neural network.
  - When trained over many images, the network learns to classify images correctly.

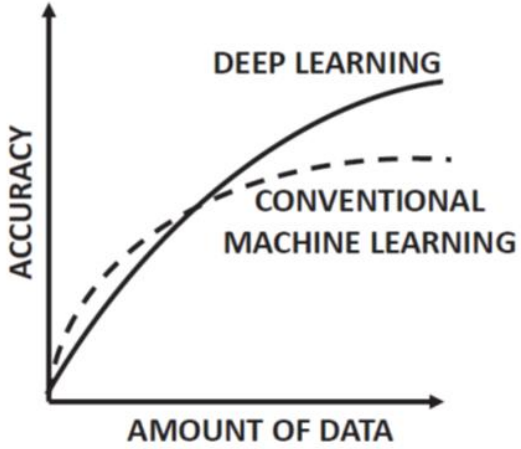
When do we prefer Deep Learning?

- For smaller data sets, traditional machine learning methods often provide slightly better performance.
- Traditional models often provide more choices, interpretable insights, and ways to handcraft features.
- For larger data sets, deep learning methods tend to dominate.



Conventional ML versus Deep Learning

- Traditional machine learning models reach their learning capacity early because they are simple neural networks
  - Logistic/linear Regression, Decision Tree, Naïve Bayes, Support Vector Machien etc. can be represented as special cases of shallow neural networks.
- When we have more data, we can add more computational units to improve the performance



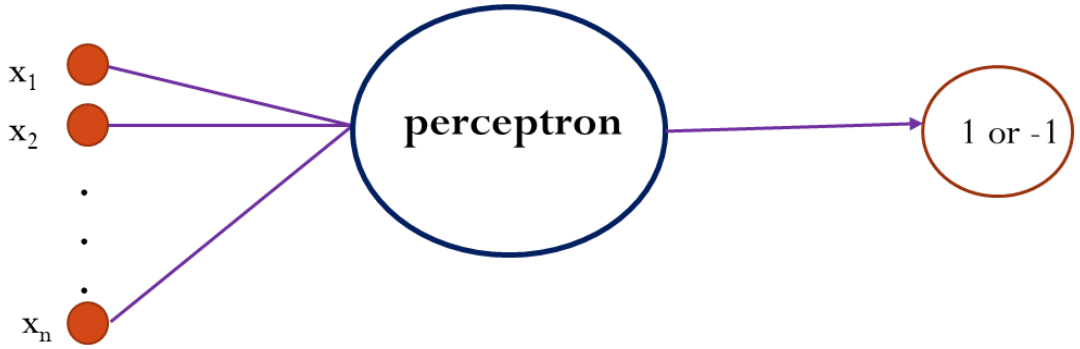
How do we choose between traditional ML or Deep Learning approaches?

- If you have less data with noise, you want to use conventional ML
  - When data points are captured well by a vector of meaningful features, non-neural ML would work well
- If you have a lot of data with rich “structure”, you want to use neural networks
  - Structure is often learned by assuming deep neural architectures

Single Layer Networks: The Perceptron

In the binary classification problem, each training example $(\boldsymbol{X}, t)$ contains feature variables $\boldsymbol{X} = (x_1, x_2, \ldots, xn)$, and label t drawn from $\{-1, +1\}$

- Given the descriptive attribute values of an instance $x_1, x_2, \ldots, xn$
- Classify the instance as one class (t = *1*) or the other class (t = *-1*)

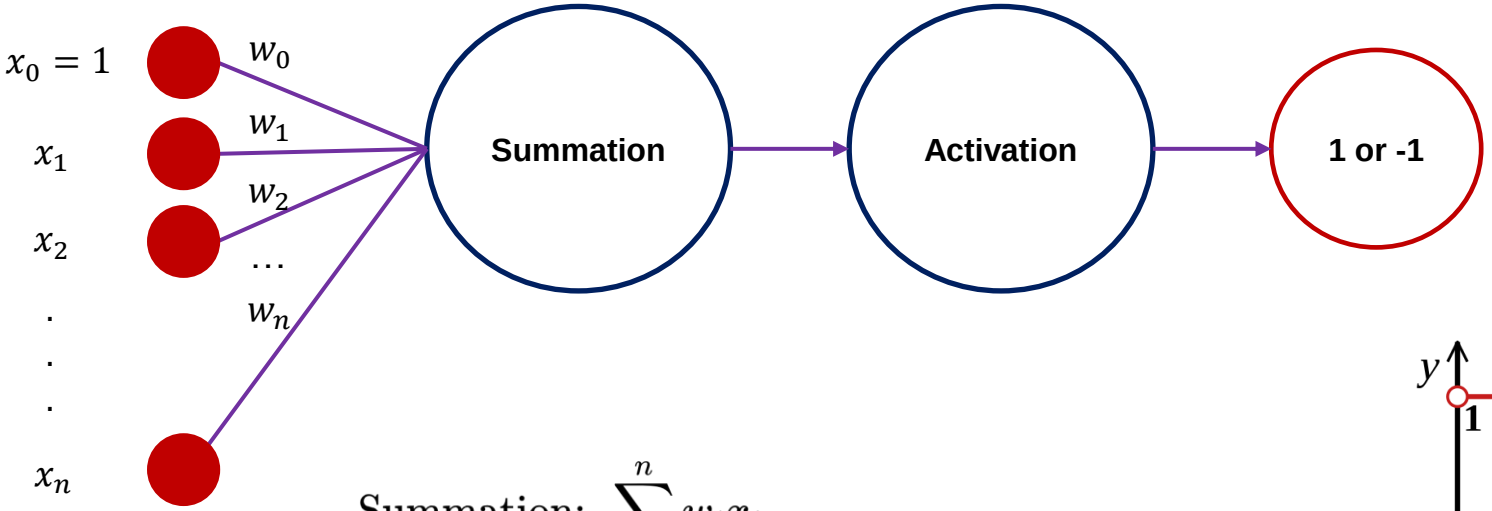


Perceptron: Understand as a Linear Model

A linear model employs a linear combination of descriptive attributes, namely decision boundary or $f(\boldsymbol{x})$, to classify instances.

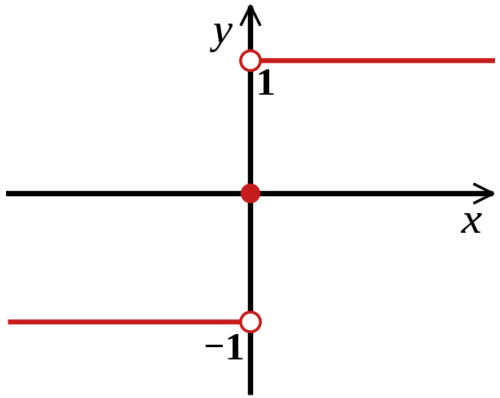
- $f(\boldsymbol{x}) = w_0 + w_1 x_1 + \cdots + w_n x_n$
- The coefficients $w_0, w_1, \ldots, w_n$ are parameters and to be learned from training data.
- A new instance (not in the training data) is classified as one class if $f(\boldsymbol{x}) > 0$ or the other class if $f(\boldsymbol{x}) \leq 0$

Perceptron: Two steps -- Sum and Activation



$$\text{Summation: } \sum_{i=0}^{n} w_i x_i$$

$$\text{Activation: } o = \begin{cases} 1 & if \sum_{i=0}^{n} w_i x_i > 0 \\ -1 & otherwise \end{cases}$$



Perceptron: A Learning Process

- Given training examples, it addresses how to learn values of $w_0, w_1, w_2, \dots, w_n$
- Considering one training example $x_1, x_2, \dots, x_n, t$ (class label)
  - Initialize the parameters $w_0, w_1, w_2, \dots, w_n$
  - Calculate $f(x) = w_0x_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n$
  - Assign perceptron output $o = 1$ if $f(x) > 0$, or $o = -1$ if $f(x) \leq 0$
  - Feedback adjustment: Update w_i for $i = 0, 1, 2, \dots, n$ as follows:

$$w_i = w_i + \eta(t - o)x_i$$

    - $(t - o)x_i$ determines the direction of updating w_i
    - η is the learning rate usually taking a small number (i.e., 0.1 or below). It is made to decay as the number of updates increases

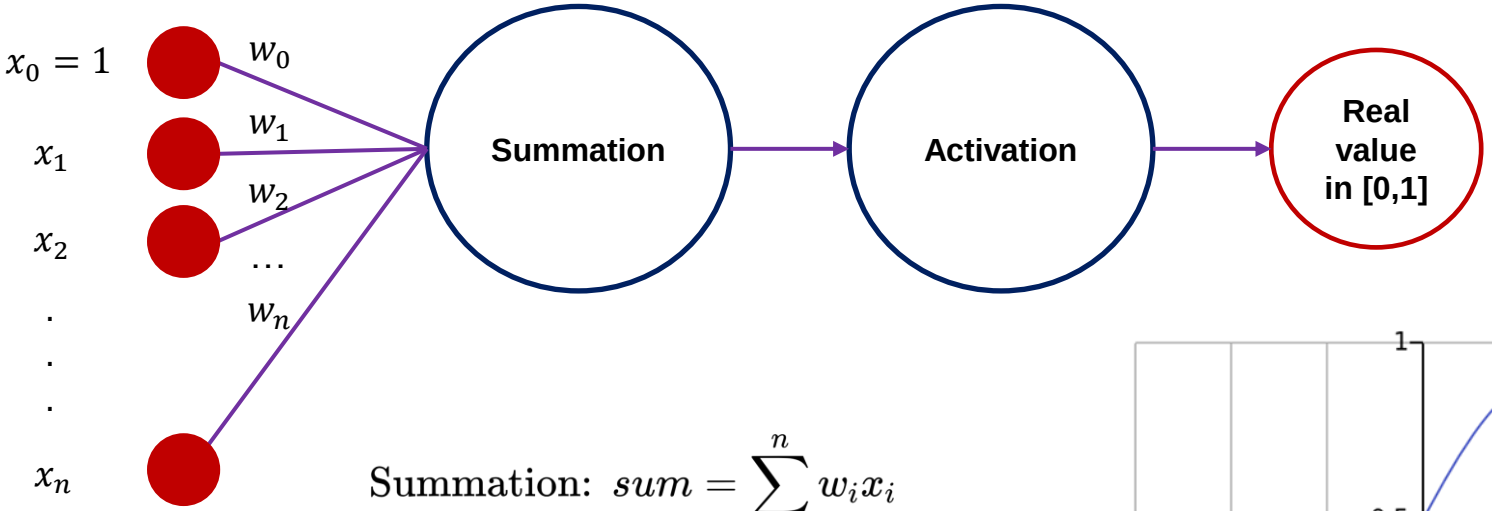
The overall learning process is described as below.

- Initially, randomly set values of $w_0, w_1, w_2, \dots, w_n$
- Go through all training examples many times to update values of $w_0, w_1, w_2, \dots, w_n$ until all training examples are correctly classified

Neural Network

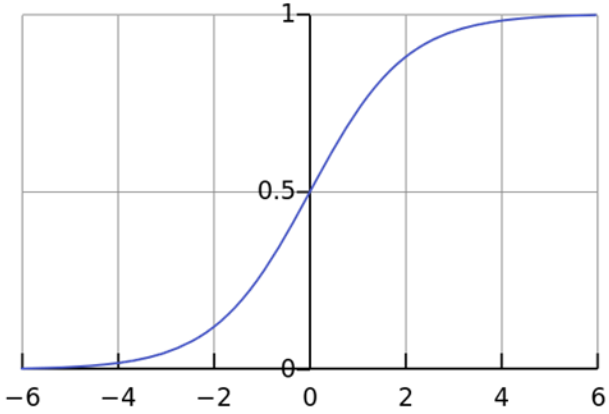
- Perceptron is a linear model and it is ineffective for classifying data sets with non-linear decision boundaries.

- Solution
  - Change the activation function to a nonlinear function (e.g., Sigmoid, ReLU)
  - Build multi-layer network (e.g., Multi-layer Perceptron)

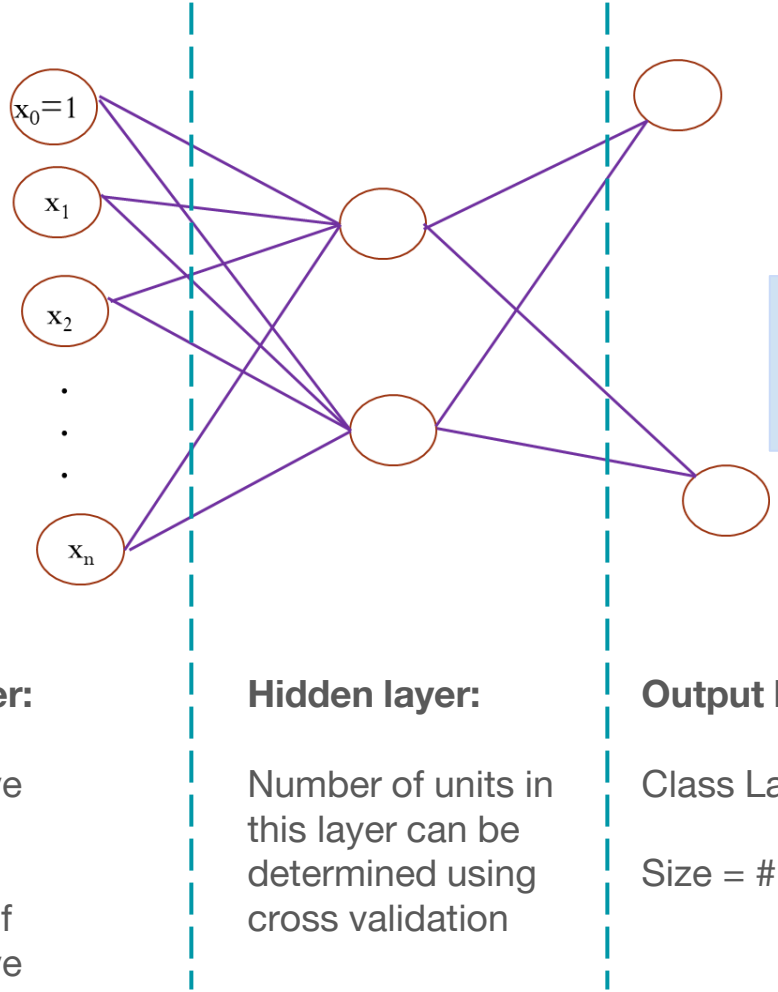


$$\text{Summation: } sum = \sum_{i=0}^{n} w_i x_i$$

$$\text{Activation: } o = \frac{1}{1 + e^{-sum}}$$



* Deep neural networks require activation function to incorporate non-linearity; in addition, it should be differentiable. Sign function is not differentiable.



Note that every unit at each layer is connected to all units from the preceding layer

Input layer:

Descriptive Attributes

Size = # of Descriptive Attributes+1

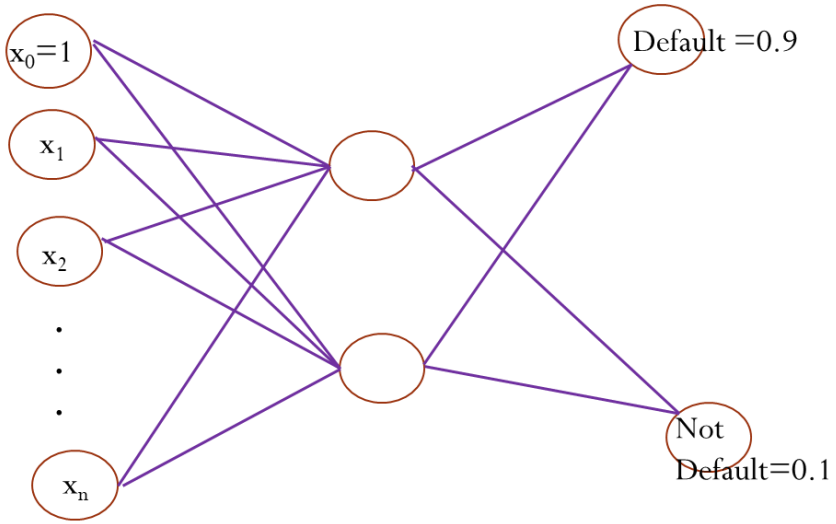
Hidden layer:

Number of units in this layer can be determined using cross validation

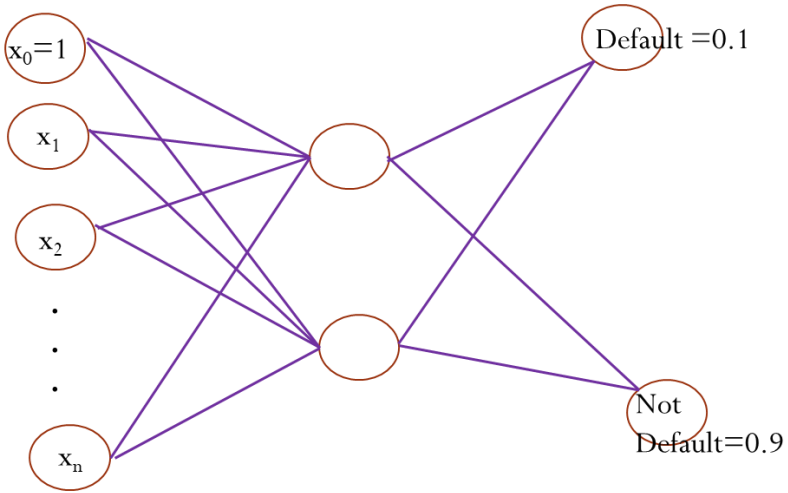
Output layer:

Class Label Attribute

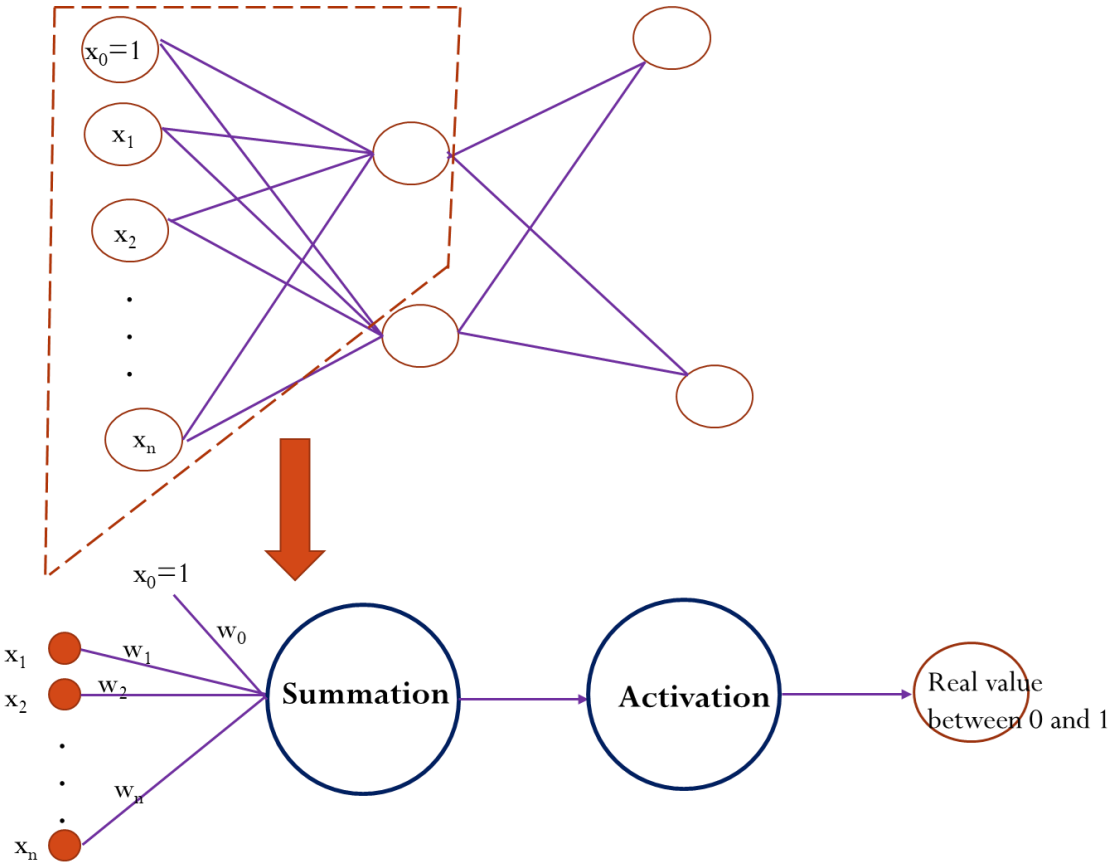
Size = # of Classes



Given a training example $<x_1, x_2, \ldots, x_n$, Default$>$



Given a training example $<x_1, x_2, \ldots, x_n$, Not Default$>$


$x_0=1$
x_1
x_2
x_n
$x_0=1$
w_0
x_1
w_1
x_2
w_2
w_n
x_n
Summation
Activation
Real value
between 0 and 1

Forward Propagation

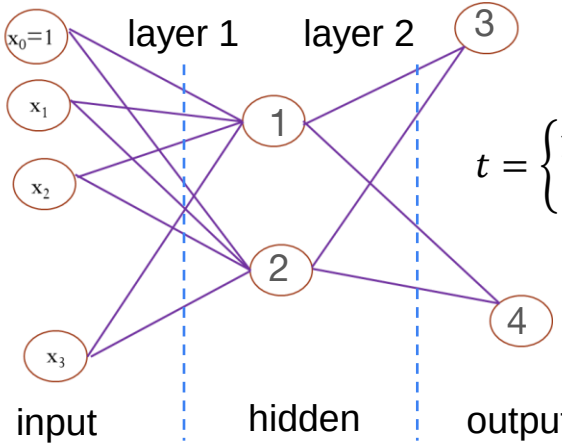
- How to use Neural Network to classify a test instance? Let us work out an example together.

$W^{[1]}$

Edge Weight (layer 1)	w_0	w_1	w_2	w_3
1	0	0.1	0.1	0.2
2	0.1	0.3	0.4	0.2

$W^{[2]}$

Edge Weight (layer 2)	w_1	w_2
3	0.9	0.1
4	0.2	0.8



$$t = \begin{cases} 1, \text{if } o^1 > o^0; \\ 0, \text{otherwise} \end{cases}$$

Data	x_0	x_1	x_2	x_3	t
	1	1	0	0	?

$o^1 = o_3$	$o^0 = o_4$
?	?

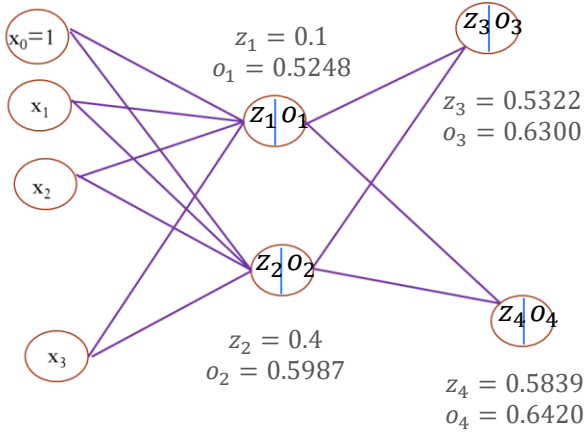
- Summation:

$$z_1 = [0, 0.1, 0.1, 0.2] \times \begin{bmatrix} x_0 = 1 \\ x_1 = 1 \\ x_2 = 0 \\ x_3 = 0 \end{bmatrix} = 0.1$$

$$z_2 = [0.1, 0.3, 0.4, 0.2] \times \begin{bmatrix} x_0 = 1 \\ x_1 = 1 \\ x_2 = 0 \\ x_3 = 0 \end{bmatrix} = 0.4$$

$$z_3 = [0.9, 0.1] \times \begin{bmatrix} o_1 = 0.5248 \\ o_2 = 0.5987 \end{bmatrix} = 0.5322$$

$$z_4 = [0.2, 0.8] \times \begin{bmatrix} o_1 = 0.5248 \\ o_2 = 0.5987 \end{bmatrix} = 0.5839$$



- Activation:

$$o_1 = \frac{1}{1+e^{-z_1}} = \frac{1}{1+e^{-0.1}} = 0.5248$$

$$o_2 = \frac{1}{1+e^{-z_2}} = \frac{1}{1+e^{-0.4}} = 0.5987$$

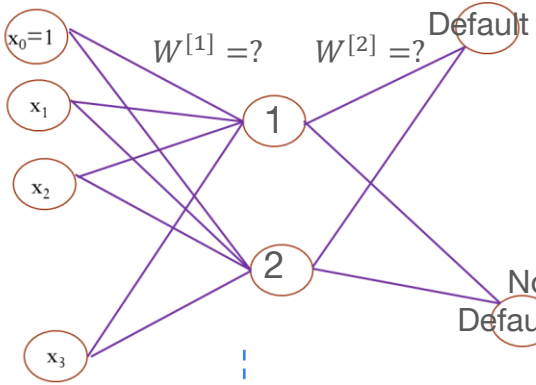
$$o_3 = \frac{1}{1+e^{-z_3}} = \frac{1}{1+e^{-0.5322}} = 0.6300$$

$$o_4 = \frac{1}{1+e^{-z_4}} = \frac{1}{1+e^{-0.5839}} = 0.6420$$

Backward Propagation

- Evaluate for the objective
  1) Assume a Loss Function L (e.g., squared error)
  2) t represents the ground truth of raw output scores for potential classes
  3) The loss for the ith training example is as follows:

$$L_i = \sum_{c \in \{1\text{: Default, } 0\text{: Not Default}\}} \frac{1}{2}(t_i^c - o_i^c)^2$$



$w \rightarrow z \rightarrow o$ | $w \rightarrow z \rightarrow o \rightarrow L$

$\Delta w \leftarrow \Delta z \leftarrow \Delta o$ | $\Delta w \leftarrow \Delta z \leftarrow \Delta o \leftarrow \Delta L$

- Learn for the updated weights W: $\boldsymbol{W^{new} = W^{old} - \eta \times \frac{\partial L}{\partial W}}$
- The chain rule for partial derivatives: $\boldsymbol{\frac{\partial L}{\partial W} = \frac{\partial L}{\partial o} \times \frac{\partial o}{\partial z} \times \frac{\partial z}{\partial W}}$

Example of Backward Propagation

Old weights are given as follows:

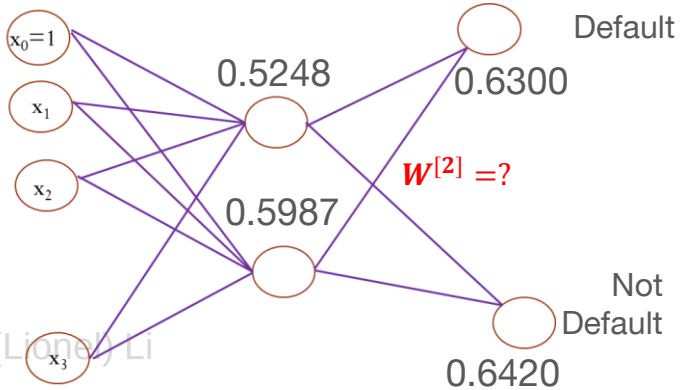
Edge Weight (layer 1)	w_0	w_1	w_2	w_3
1	0	0.1	0.1	0.2
2	0.1	0.3	0.4	0.2

Edge Weight (layer 2)	w_1	w_2
3	0.9	0.1
4	0.2	0.8

Suppose label of training example is

Data	x_0	x_1	x_2	x_3	t(1)	t(0)
	1	1	0	0	0.01	0.99

Current state of the neural network:



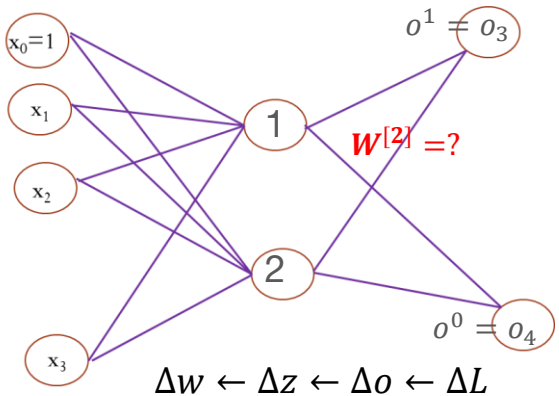
Computing details

1. Partial derivative of the loss function

$$\frac{\partial L}{\partial o^1} = \frac{\partial}{\partial o^1}\left(\frac{1}{2}\left(t^1 - o^1\right)^2 + \frac{1}{2}\left(t^0 - o^0\right)^2\right) = -\left(t^1 - o^1\right) = -(0.01 - 0.6300) = 0.6200$$

$$\frac{\partial L}{\partial o^0} = \frac{\partial}{\partial o^0}\left(\frac{1}{2}\left(t^1 - o^1\right)^2 + \frac{1}{2}\left(t^0 - o^0\right)^2\right) = -\left(t^0 - o^0\right) = -(0.99 - 0.6420) = -0.3480$$

Note: define the output node $o^1 := o_3$ and $o^0 := o_4$ in the example network



$\Delta w \leftarrow \Delta z \leftarrow \Delta o \leftarrow \Delta L$

2. Partial derivative of the activation function

$$\frac{\partial o^1}{\partial z_3} = \frac{\partial}{\partial z}\left(\frac{1}{1+e^{-z_3}}\right) = \frac{1}{1+e^{-z_3}} \times \left(1 - \frac{1}{1+e^{-z_3}}\right) = 0.6300 \times (1 - 0.6300) = 0.2331$$

$$\frac{\partial o^0}{\partial z_4} = \frac{\partial}{\partial z}\left(\frac{1}{1+e^{-z_4}}\right) = \frac{1}{1+e^{-z_4}} \times \left(1 - \frac{1}{1+e^{-z_4}}\right) = 0.6420 \times (1 - 0.6420) = 0.2298$$

Assume a sigmoid function

$$f(x) = \frac{1}{1+e^{-x}},$$

its first derivative is

$$f'(x) = \frac{\partial}{\partial x}\left(\frac{1}{1+e^{-x}}\right)$$

$$= 1 - \frac{1}{1+e^{-x}}$$

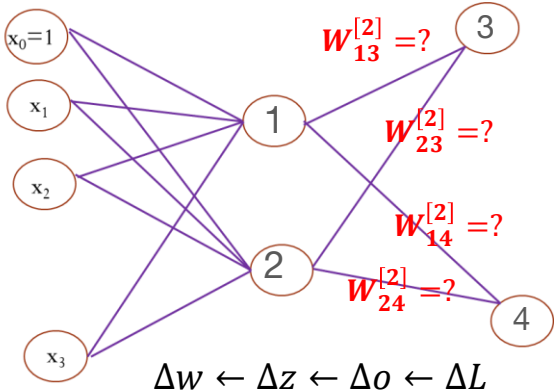
3. Partial derivative of the (weighted) summation

$$\frac{\partial z_3}{\partial w_{13}^{[2]}} = \frac{\partial}{\partial w_{13}^{[2]}}\left(w_{13}^{[2]} \times o_1 + w_{23}^{[2]} \times o_2\right) = o_1 = 0.5248$$

$$\frac{\partial z_3}{\partial w_{23}^{[2]}} = \frac{\partial}{\partial w_{23}^{[2]}}\left(w_{13}^{[2]} \times o_1 + w_{23}^{[2]} \times o_2\right) = o_2 = 0.5987$$

$$\frac{\partial z_4}{\partial w_{14}^{[2]}} = \frac{\partial}{\partial w_{14}^{[2]}}\left(w_{14}^{[2]} \times o_1 + w_{24}^{[2]} \times o_2\right) = o_1 = 0.5248$$

$$\frac{\partial z_4}{\partial w_{24}^{[2]}} = \frac{\partial}{\partial w_{24}^{[2]}}\left(w_{14}^{[2]} \times o_1 + w_{24}^{[2]} \times o_2\right) = o_2 = 0.5987$$



$\Delta w \leftarrow \Delta z \leftarrow \Delta o \leftarrow \Delta L$

4. Assume learning rate $\eta = 0.1$; the updated weights are

$$w_{13}^{[2]} = 0.9 - \eta \times \frac{\partial L}{\partial o^1} \times \frac{\partial o^1}{\partial z_3} \times \frac{\partial z_3}{\partial w_{13}^{[2]}} = 0.9 - 0.1 \times 0.6200 \times 0.2331 \times 0.5248 = 0.8924$$

$$w_{23}^{[2]} = 0.1 - \eta \times \frac{\partial L}{\partial o^1} \times \frac{\partial o^1}{\partial z_3} \times \frac{\partial z_3}{\partial w_{23}^{[2]}} = 0.1 - 0.1 \times 0.6200 \times 0.2331 \times 0.5987 = 0.0913$$

$$w_{14}^{[2]} = 0.2 - \eta \times \frac{\partial L}{\partial o^0} \times \frac{\partial o^0}{\partial z_4} \times \frac{\partial z_4}{\partial w_{14}^{[2]}} = 0.2 - 0.1 \times (-0.3480) \times 0.2298 \times 0.5248 = 0.2042$$

$$w_{24}^{[2]} = 0.8 - \eta \times \frac{\partial L}{\partial o^0} \times \frac{\partial o^0}{\partial z_4} \times \frac{\partial z_4}{\partial w_{24}^{[2]}} = 0.8 - 0.1 \times (-0.3480) \times 0.2298 \times 0.5987 = 0.8048$$

Old Edge Weight (layer 2)	w_1	w_2
3	0.9	0.1
4	0.2	0.8

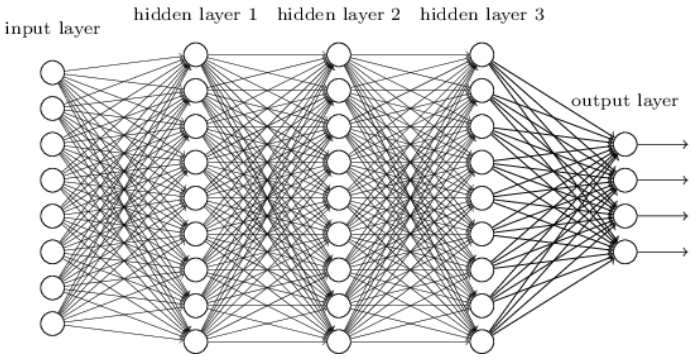
New Edge Weight (layer 2)	w_1	w_2
3	0.8924	0.0913
4	0.2042	0.8048

- Learning a Neural Network from training examples is to learn weights of all edges in a Neural Network.
- Overall Learning Process:
  1. For each training example, calculate its outputs using a Neural Network - Forward Propagation;
  2. Update weights based on the differences between calculated outputs and targets - Backward Propagation;
  3. Repeat the process for N (e.g., 500) times - Iterative Updating

Discussion: Modern Deep Learning Models

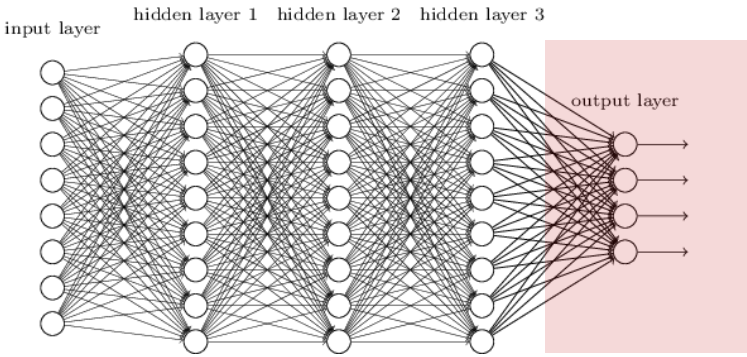
Building up from the in-class example to realistic sizable DL models

- Class #
- Loss function
- Hidden layers # and structure
- Activation function
- Learning process and techniques



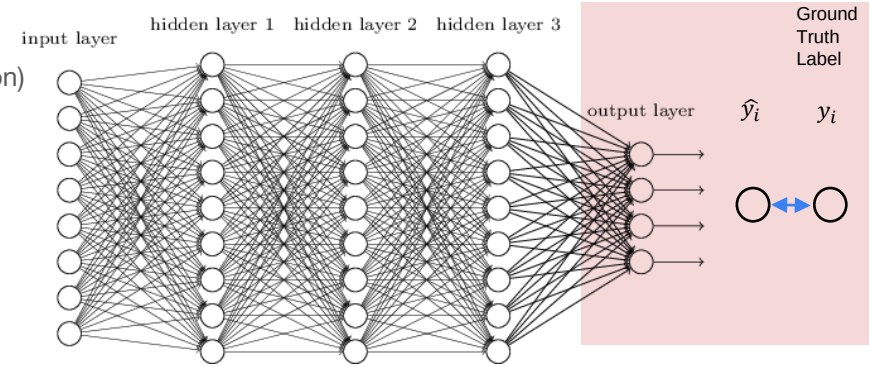
Discussion: Modern Deep Learning Models

- Class #
  - Over one thousand classes vs. two classes as shown
- Loss function
  - Cross-entropy loss vs. MSE
- Hidden layers # & structure
  - Over one-hundred hidden layers vs. one hidden layer as shown
  - More advanced model architecture, e.g., transformer
- Activation function
  - ReLU (Rectified Linear Unit) vs. Sigmoid
- Learning process and techniques
  - Batch training vs. single training example
  - Batch Normal
  - Random Dropout



Discussion: Modern Deep Learning Models

- Class #
  - Over one thousand classes vs. two classes as shown
- Loss function
  - Cross-entropy loss (multi-class classification) vs. MSE (regression)
- Hidden layers # & structure
  - Over one-hundred hidden layers vs. one hidden layer as shown
  - More advanced model architecture, e.g., transformer
- Activation function
  - ReLU (Rectified Linear Unit) vs. Sigmoid
- Learning process and techniques
  - Batch training vs. single training example
  - Batch Norm
  - Random Dropout



Mean Square Error

$$MSE = \frac{1}{n}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Penalizes larger Margin Of Errors (MOE)

Use for: Regression

Binary Cross-Entropy

$$BCE = -\frac{1}{n}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

Penalizes confident, wrong predictions

Use for: Binary Classification

Multi-class Cross-Entropy

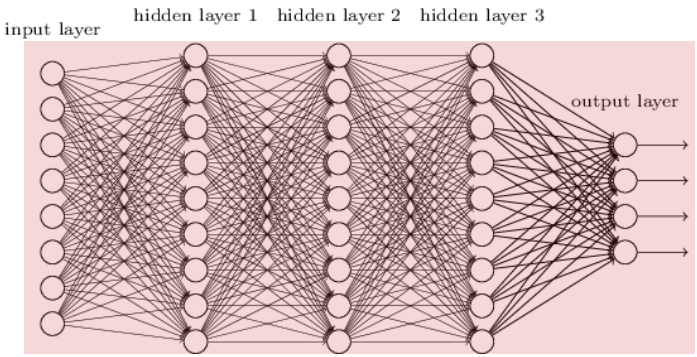
$$CE = -\sum_{i=1}^{N} y_i \log(\hat{y}_i)$$

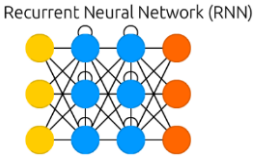
Penalizes misclassified predictions

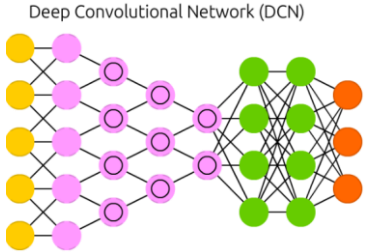
Use for: Multi-class Classification

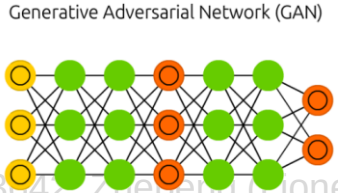
Discussion: Modern Deep Learning Models

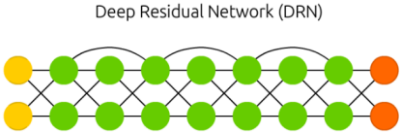
- Class #
  - Over one thousand classes vs. two classes as shown
- Loss function
  - Cross-entropy loss vs. MSE
- Hidden layers # & structure
  - Over one-hundred hidden layers vs. one hidden layer as shown
  - More advanced model architecture, e.g., transformer
- Activation function
  - ReLU (Rectified Linear Unit) vs. Sigmoid
- Learning process and techniques
  - Batch training vs. single training example
  - Batch Norm
  - Random Dropout

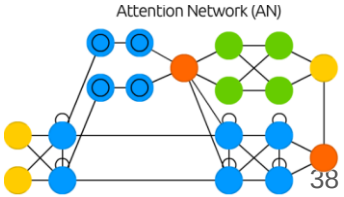






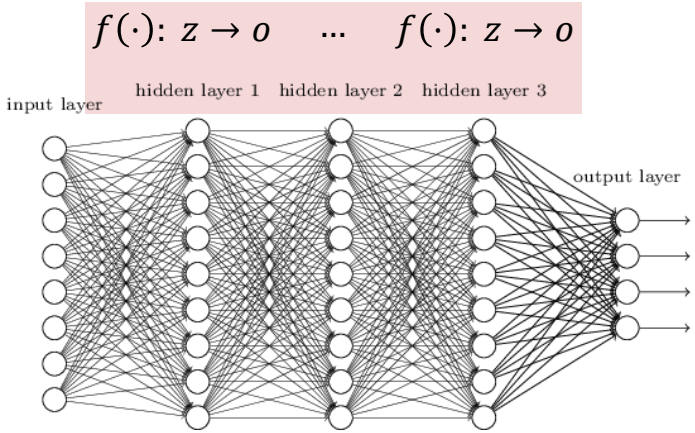


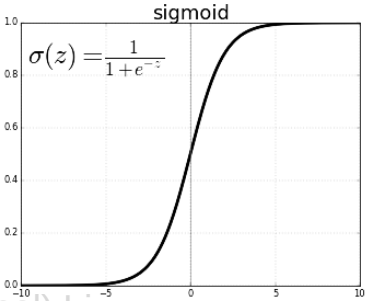


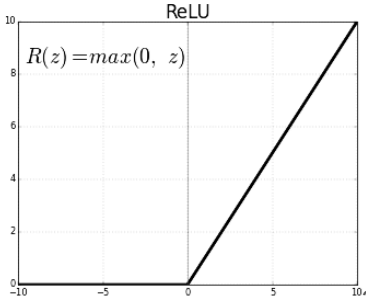


Discussion: Modern Deep Learning Models

- Class #
  - Over one thousand classes vs. two classes as shown
- Loss function
  - Cross-entropy loss vs. MSE
- Hidden layers # & structure
  - Over one-hundred hidden layers vs. one hidden layer as shown
  - More advanced model architecture, e.g., transformer
- Activation function
  - ReLU (Rectified Linear Unit) vs. Sigmoid
- Learning process and techniques
  - Batch training vs. single training example
  - Batch Norm
  - Random Dropout

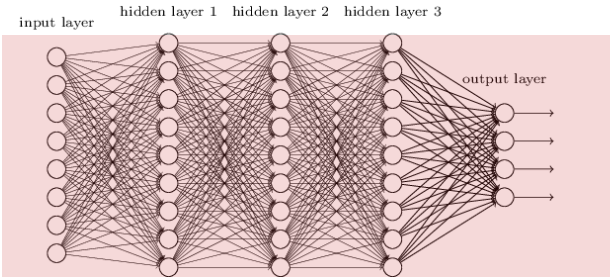


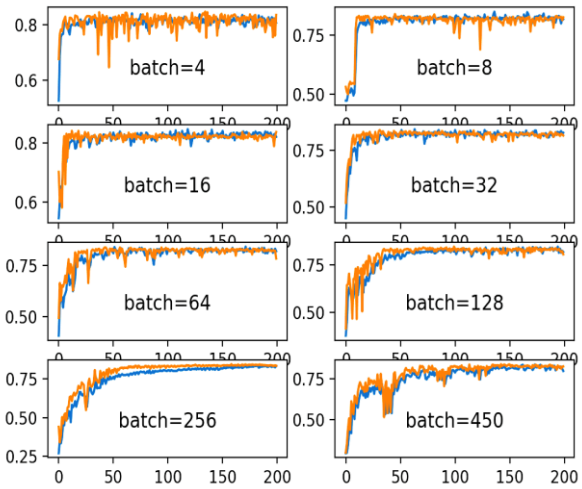




Discussion: Modern Deep Learning Models

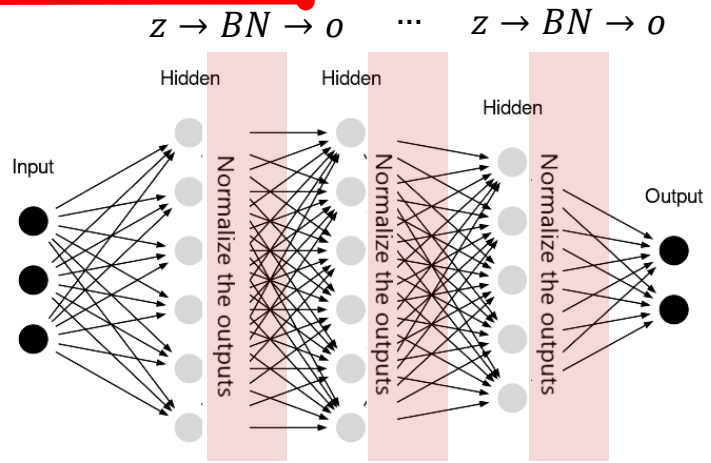
- Class #
  - Over one thousand classes vs. two classes as shown
- Loss function
  - Cross-entropy loss vs. MSE
- Hidden layers # & structure
  - Over one-hundred hidden layers vs. one hidden layer as shown
  - More advanced model architecture, e.g., transformer
- Activation function
  - ReLU (Rectified Linear Unit) vs. Sigmoid
- Learning process & techniques
  - Batch training (batch size = m) vs. single training example (batch size =1)
  - Batch Norm
  - Random Dropout



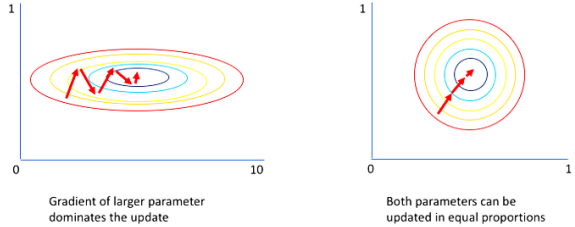


Discussion: Modern Deep Learning Models

- Class #
  - Over one thousand classes vs. two classes as shown
- Loss function
  - Cross-entropy loss vs. MSE
- Hidden layers # & structure
  - Over one-hundred hidden layers vs. one hidden layer as shown
  - More advanced model architecture, e.g., transformer
- Activation function
  - ReLU (Rectified Linear Unit) vs. Sigmoid
- Learning process & techniques
  - Batch training (batch size = m) vs. single training example (batch size =1)
  - Batch Norm (regularize data distribution $\mu = 0, \sigma = 1$)
  - Random Dropout

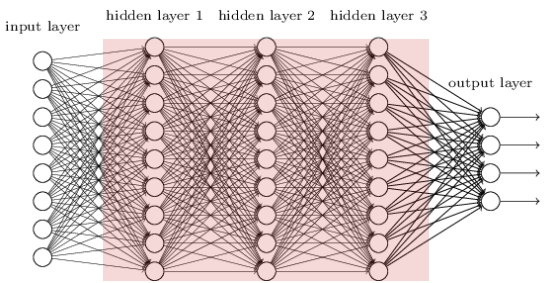


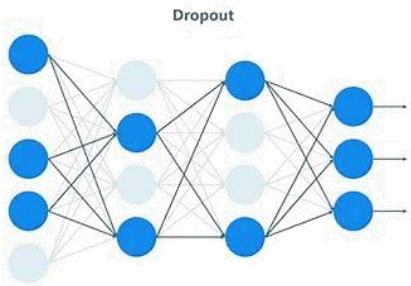
Why normalize?

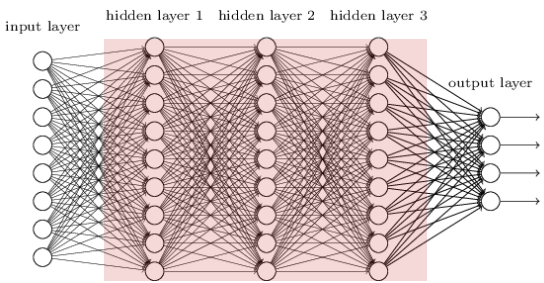
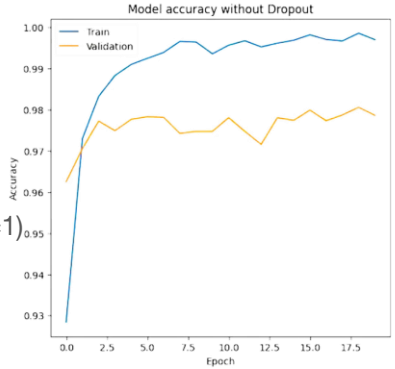
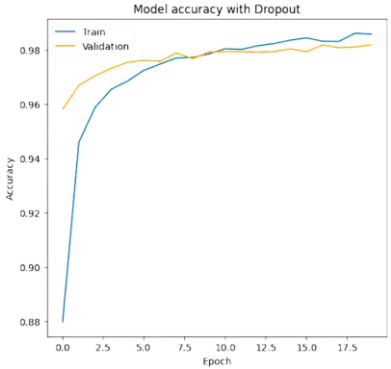


Discussion: Modern Deep Learning Models

- Class #
  - Over one thousand classes vs. two classes as shown
- Loss function
  - Cross-entropy loss vs. MSE
- Hidden layers # & structure
  - Over one-hundred hidden layers vs. one hidden layer as shown
  - More advanced model architecture, e.g., transformer
- Activation function
  - ReLU (Rectified Linear Unit) vs. Sigmoid
- Learning process & techniques
  - Batch training (batch size = m) vs. single training example (batch size =1)
  - Batch Norm (regularize data distribution $\mu = 0, \sigma = 1$)
  - Random Dropout (decrease the reliance on specific neurons)





Discussion: Modern Deep Learning Models

- Class #
  - Over one thousand classes vs. two classes as shown
- Loss function
  - Cross-entropy loss vs. MSE
- Hidden layers # & structure
  - Over one-hundred hidden layers vs. one hidden layer as shown
  - More advanced model architecture, e.g., transformer
- Activation function
  - ReLU (Rectified Linear Unit) vs. Sigmoid
- Learning process & techniques
  - Batch training (batch size = m) vs. single training example (batch size =1)
  - Batch Norm (regularize data distribution $\mu = 0, \sigma = 1$)
  - Random Dropout (decrease the reliance on specific neurons)